


# INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

**COURSE TITLE:** BACHELOR OF TECHNOLOGY

**SUBJECT:** PYTHON PROGRAMMING LABORATORY

**PROJECT TITLE:** CAMPUS ENERGY USAGE ANALYZER
Electricity Consumption Monitoring and Analytics System

**NAME:** 1. _____ 2. _____

3. _____ 4. _____

5. _____

**ROLL NO:** _____

**ENROLLMENT NO:** _____

**SECTION:** _____

**SUBMITTED TO:** _____

**DEPARTMENT NAME:** ELECTRONICS AND COMMUNICATION ENGINEERING

**ACADEMIC SESSION:** 2026-2027

**DATE OF SUBMISSION:** _____

## Abstract

This project develops an offline Python desktop application that analyses hourly electricity consumption of campus buildings. It validates CSV records, calculates energy totals, average and peak demand, load factor, building shares and flat-tariff cost estimates, flags statistically unusual readings with an interquartile-range (IQR) rule, and presents eight Matplotlib charts inside a Tkinter dashboard with CSV, JSON and PDF export. The bundled demonstration dataset is **synthetic**: 3,600 hourly records for five buildings over 30 days generated with a fixed random seed; it is not real meter data. On this dataset the campus used 62,035.5 kWh (2,067.8 kWh/day), the maximum 1-hour average demand was 242.1 kW, the load factor was 35.6% and the estimated cost at an illustrative ₹8/kWh was ₹496,283.67. Computer Laboratory had the largest share (27.1%). Comparing readings only with the same building, day type and hour detected all 8 injected test anomalies with 12 additional flags, while a single campus-wide threshold detected 4. The automated test suite ran 73 tests (73 passed, 0 failed, 0 skipped).

**Keywords:** electricity consumption; kWh; peak demand; load factor; IQR outlier detection; Python; Pandas; NumPy; Matplotlib; Tkinter; data validation.

## Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Literature Review
4. Software Requirements
5. System Architecture
6. Methodology and Flowcharts
7. Dataset Design and Simulation
8. Mathematical Formulations
9. Implementation
10. Results and Observations
11. Testing and Validation
12. Discussion
13. Limitations
14. Applications and Future Scope
15. Conclusion
16. References
Appendix A: Installation and Execution
Appendix B: Team Contributions

## 1. Introduction

Educational campuses run classrooms, laboratories, libraries and offices whose electricity use differs by hour, weekday and purpose. When readings are kept only as spreadsheets it is difficult to see which building uses most energy, when the campus peak occurs, how much the energy is likely to cost, and which readings are unusual. This project builds a small, transparent analytics tool for such interval data. It is 100% software: no meter hardware or live connection is involved, so the application analyses either simulated data or CSV files supplied by the user.

## 2. Problem Statement and Objectives

**Problem statement.** Analyse electricity-meter readings for several campus buildings: use functions to flag abnormal and negative readings, lists/dictionaries and CSV for building records, a *MeterRecord* class, NumPy for totals and moving averages, Pandas for summaries and JSON export, and Matplotlib for usage trends; then compare buildings and judge whether spikes reflect real use or incomplete data.

- Validate imported records and explain every rejected row; never replace missing values with zero.
- Generate a reproducible synthetic dataset (fixed seed) with realistic building profiles and labelled anomalies.
- Compute total and daily energy, average power, peak interval demand, load factor, shares and costs.
- Detect unusual readings with an IQR rule applied to comparable groups and evaluate it against an alternative.
- Present KPIs, eight charts and tables in a Tkinter dashboard that refreshes on every filter change.
- Export filtered records, building summaries, anomaly reports, monthly JSON and a PDF report.
- Verify the logic with automated tests and document all limitations honestly.

## 3. Literature Review

Interval (typically 15-60 minute) meter data are the standard basis for building-energy analysis; daily totals, hour-of-day profiles and weekday/weekend comparisons are common first steps because building loads follow occupancy schedules [6]. Tukey's box-plot rule, which marks values beyond 1.5 IQR from the quartiles, is a simple, distribution-free outlier test [1]. Surveys of anomaly detection stress

that an anomaly is defined relative to a context: a value normal at 14:00 on a weekday may be abnormal at 02:00 [2]. This motivates contextual groups instead of one campus-wide threshold. Load factor (average/peak demand) is a standard utilisation measure in power systems, and India's Energy Conservation Building Code encourages metering and monitoring of building loads [6]. The implementation relies on NumPy [3], Pandas [4], Matplotlib [5], Tkinter [7], ReportLab [8] and pytest [9].

## 4. Software Requirements

*Table 1: Software requirements*

| Component | Purpose in this project |
|---|---|
| Python 3.10+ (tested on 3.12) | Core language; dataclasses, pathlib, logging, csv/json |
| Tkinter / ttk | Desktop GUI: sidebar, filters, KPI cards, notebooks, Treeview tables, dialogs |
| Pandas ≥ 2.2 | DataFrames, validation, group-by summaries, pivot tables, date handling |
| NumPy ≥ 1.26 | Percentiles (quartiles), moving average (convolution), random simulation |
| Matplotlib ≥ 3.8 | Eight charts embedded with FigureCanvasTkAgg (+ zoom/pan/save toolbar) |
| ReportLab ≥ 4.0 | PDF analysis report export and this project report |
| pytest / pytest-cov | Automated unit and GUI tests, coverage measurement |
| OS / hardware | Windows 10/11 (target), also runs on Linux; no special hardware, works offline |

## 5. System Architecture

The code is organised in four layers (Figure 1). Lower layers never import higher layers, which prevents circular imports and lets all calculations be tested without a GUI. *config.py* holds every tunable constant (tariff, IQR multiplier, default buildings, colours and portable paths).

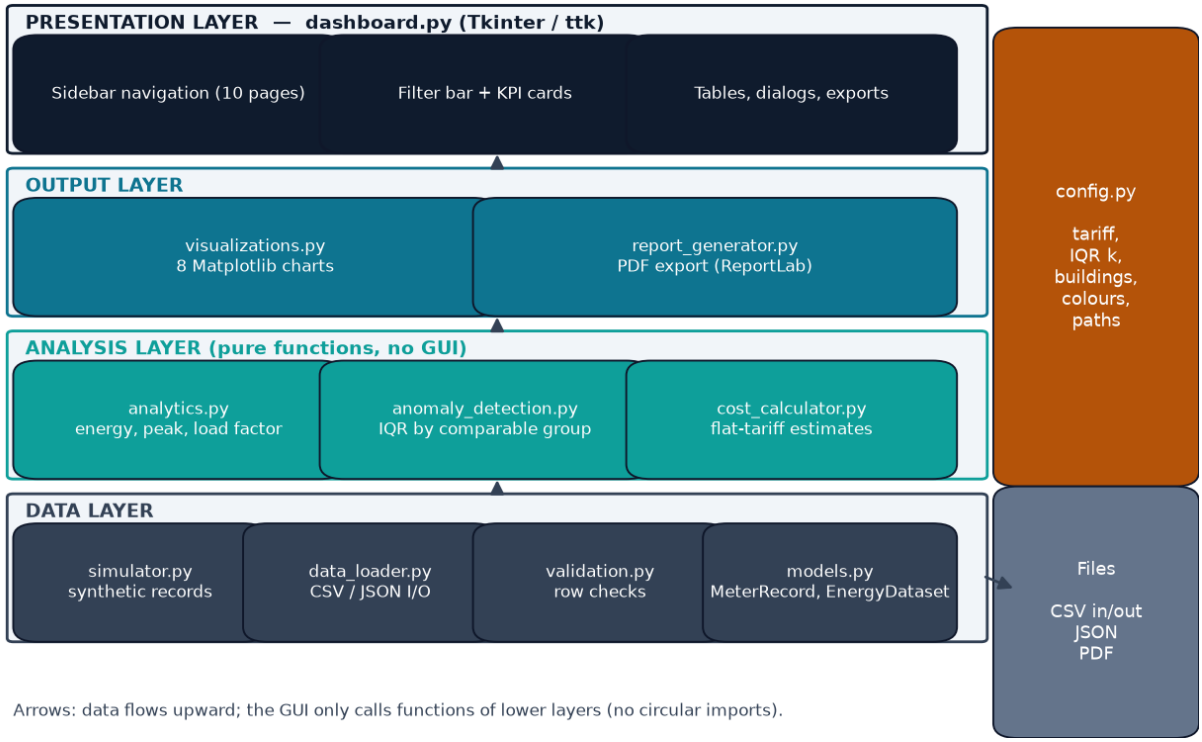



*Figure 1: System architecture (modules in src/campus_energy/)*

## 6. Methodology and Flowcharts

Data enter either from the simulator or from a CSV file read entirely as text, so that the original values of rejected rows can be shown unchanged (Figure 2). Accepted rows form the working dataset. Anomaly thresholds are computed once on the whole dataset so that they do not change when the user zooms into a few days; building and date filters then select what is displayed. Figure 3 shows the energy-analysis workflow that runs after every change.

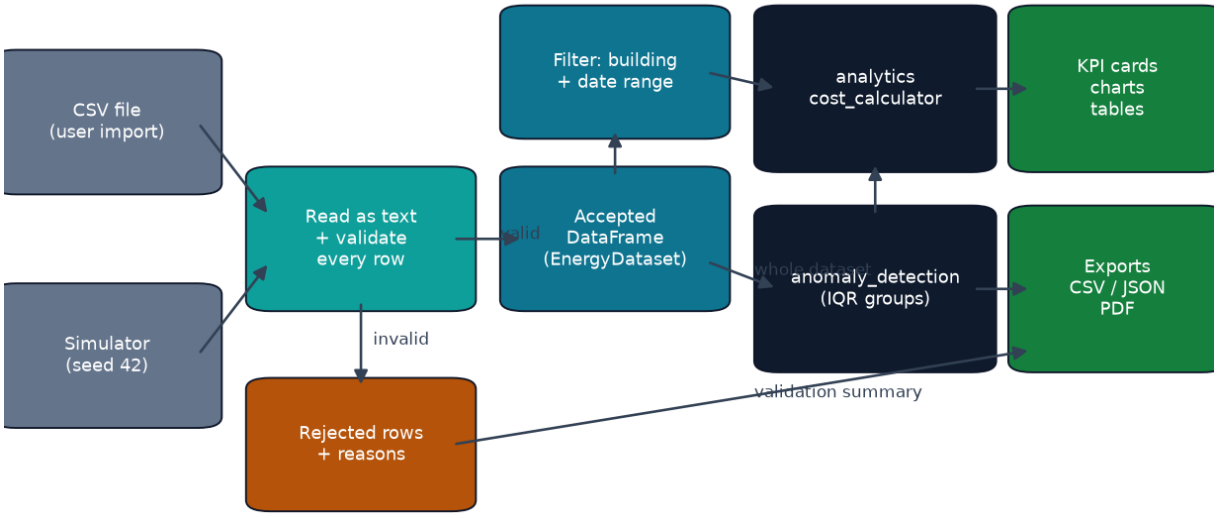



*Figure 2: Data-processing flow from input to outputs*

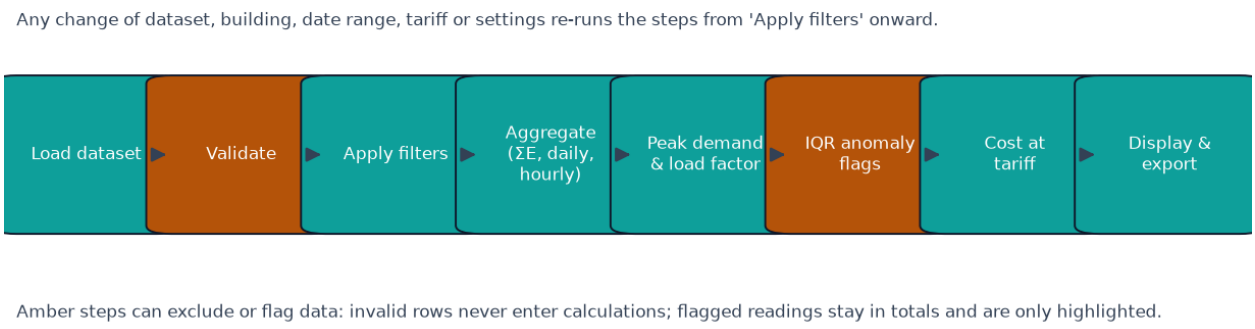



*Figure 3: Energy-analysis workflow*

## 7. Dataset Design and Simulation

Each record describes one building over one interval. Required CSV columns are listed in the schema table; *estimated_cost_inr* is always recomputed from the current tariff and *data_source* marks every simulated row as *synthetic*.

*Table 2: Dataset schema*

| Column | Type / unit | Meaning |
|---|---|---|
| timestamp | YYYY-MM-DD HH:MM:SS | Start of the interval |
| building_id / building_name | text | Permanent key (B001...) and display name |
| interval_hours | h (0 < t ≤ 24) | Interval length; 1 h in the sample |
| average_power_kw | kW | Average power during the interval |
| energy_kwh | kWh | Energy = average power × interval |
| estimated_cost_inr, data_source | ₹, text | Optional: cost (recomputed) and origin label |

The simulator (seed 42) models each hour as $P = P_{base} + (P_{peak} - P_{base}) \times$ activity, multiplied by Gaussian noise N(1, σ) clipped to 0.7-1.3 so power is never negative. Activity is 1 in the building's active hours, 0.35 in the hour before and after, 0.8 at 13:00 (lunch), 0 otherwise, and is scaled by a weekend factor. Default: 5 buildings × 30 days × 24 h = 3,600 records from 1 to 30 October 2026.

*Table 3: Synthetic load profiles*

| Building (profile) | Base kW | Peak kW | Active hours | Weekend factor | σ |
|---|---|---|---|---|---|
| Academic Block | 8 | 55 | 09:00-17:00 | 0.15 | 0.06 |
| Electronics Laboratory | 5 | 42 | 10:00-17:00 | 0.1 | 0.08 |
| Computer Laboratory | 12 | 48 | 09:00-18:00 | 0.25 | 0.06 |
| Central Library | 6 | 30 | 08:00-21:00 | 0.6 | 0.04 |
| Administrative Building | 4 | 26 | 09:00-17:00 | 0.08 | 0.06 |

*Table 4: Labelled anomalies injected for testing (column injected_anomaly)*

| Injected anomaly | Building | Time | Set to |
|---|---|---|---|
| Unexpected night-time use | Academic Block | Mon 05 Oct 02:00 | 0.55 × peak kW |
| Unexpected night-time use | Academic Block | Mon 19 Oct 23:00 | 0.5 × peak kW |
| Abnormally high lab use | Electronics Laboratory | Sat 10 Oct 14:00 | 1.9 × peak kW |
| Abnormally high lab use | Electronics Laboratory | Fri 23 Oct 11:00 | 1.8 × peak kW |
| Sudden demand increase | Computer Laboratory | Tue 13 Oct 03:00 | 1.2 × peak kW |
| Sudden demand increase | Computer Laboratory | Mon 26 Oct 15:00 | 1.75 × peak kW |
| Unusually low reading | Central Library | Fri 16 Oct 12:00 | 0.03 × peak kW |
| Unusually low reading | Administrative Building | Thu 08 Oct 11:00 | 0.02 × peak kW |

## 8. Mathematical Formulations

Symbols: $P_i$ = average power of interval i (kW); $t_i$ = interval length (h); $E_i$ = energy of interval i (kWh); T = tariff (₹/kWh). Assumptions: readings are interval averages, intervals do not overlap, and costs use a flat tariff only.

**(1) Interval energy.** $E_i = P_i \times t_i$ [kW × h = kWh]. Example (Academic Block, 01 Oct 10:00): 57.902 kW × 1 h = 57.902 kWh.

**(2) Total energy.** $E_{total} = \Sigma E_i$ = 62,035.459 kWh for the sample.

**(3) Average daily energy** = $E_{total}$ / D, where D = number of *days with at least one valid reading* (not the calendar span): 62,035.5 / 30 = 2,067.8 kWh/day.

**(4) Average power** = $E_{total}$ / total observed duration [kWh/h = kW]. For several buildings, the campus demand $P_c(t) = \Sigma_b P_b(t)$ is formed only at timestamps where every selected building reports; average power = mean $P_c$ = 86.16 kW.

**(5) Maximum recorded demand** $P_{max}$ = max $P_c(t)$ = 242.11 kW at 26 Oct 15:00. This is the highest *1-hour interval-average* demand, not an instantaneous peak and not a utility's billing demand (often a 15/30-minute maximum).

**(6) Load factor** LF = $P_{avg}$ / $P_{max}$ × 100% = 86.16 / 242.11 = 35.6%. Computed only with uniform intervals and at least 24 complete timestamps; with hourly data it overestimates LF relative to shorter billing intervals.

**(7) Estimated cost** C = E × T. Example from the specification: 250 kWh × ₹8/kWh = ₹2,000. ₹8/kWh is an illustrative, user-configurable value, not an official tariff; fixed charges, demand charges, taxes and time-of-day rates are excluded.

**(8) Building energy share** = $E_b$ / $E_{total}$ × 100%; e.g. Computer Laboratory: 16,806.5 / 62,035.5 = 27.09%.

**(9) Percentage change** = ($E_{current} - E_{previous}$) / $E_{previous}$ × 100%; undefined (shown as n/a) when $E_{previous}$ = 0. Example: 16-30 Oct vs 1-15 Oct = (30,984.1 − 31,051.3) / 31,051.3 = -0.22%.

**(10) IQR rule.** IQR = Q3 − Q1; lower = Q1 − k·IQR′, upper = Q3 + k·IQR′ with k = 1.5 and IQR′ = max(IQR, 0.10 × median). Quartiles use linear interpolation (NumPy default). Groups are building × day type (weekday/weekend) × hour; groups with fewer than 8 readings fall back to building × day type, then building; fewer than 4 readings are not evaluated. The 10% noise floor stops tightly clustered night loads from flagging ordinary ±5% variation; setting it to 0 gives the textbook rule.

## 9. Implementation

*Table 5: Major modules*

| Module | Responsibility |
|---|---|
| models.py | Building, MeterRecord (dataclasses), BuildingRegistry (dict), EnergyDataset (__len__, __iter__, __getitem__) |
| simulator.py | LoadProfile, generate_dataset(): deterministic synthetic records + injected anomalies |
| validation.py | validate_dataframe(): eleven row checks, rejection reasons, ValidationResult |
| data_loader.py | load_csv(), export_csv(), export_json(), export_validation_summary() |
| analytics.py | filter_data(), totals, campus demand, peak, load factor, summaries, moving_average(), kpis() |
| anomaly_detection.py | iqr_bounds(), detect_anomalies(), evaluate_against_injected() |
| cost_calculator.py | validate_tariff(), energy_cost(), building/daily/monthly costs |
| visualizations.py | Eight chart functions drawing on a Matplotlib Figure (no pyplot state) |
| report_generator.py | PDF analysis report of the current selection |
| dashboard.py / main.py | Tkinter application (10 pages) and entry point python -m campus_energy.main |

**Special methods (BL6).** *EnergyDataset* wraps a Pandas DataFrame and implements *__len__*, *__iter__* and *__getitem__*, so len(ds) gives the record count, a for-loop yields *MeterRecord* objects and ds[0] returns the first reading. This keeps fast vectorised Pandas calculations while offering a readable object view of one reading, which is what a student explains in a viva. *BuildingRegistry* also implements *__len__*, *__iter__* and *__contains__*.

**GUI.** The window has a sidebar with ten pages, a header with a dataset status indicator, and a filter bar (building, from/to date, apply, reset, export). Each page re-draws only when it is shown after a change. All actions that modify data are ordinary methods (e.g. import_csv(), remove_building(), set_tariff()), so the button handlers only collect input; this design made automated GUI tests possible. Deleting a building asks for confirmation and states that its records are removed from the working dataset in memory while the source CSV is unchanged. Keyboard shortcuts: Ctrl+1..9/0 pages, F5 refresh, Ctrl+O import, Ctrl+E export.

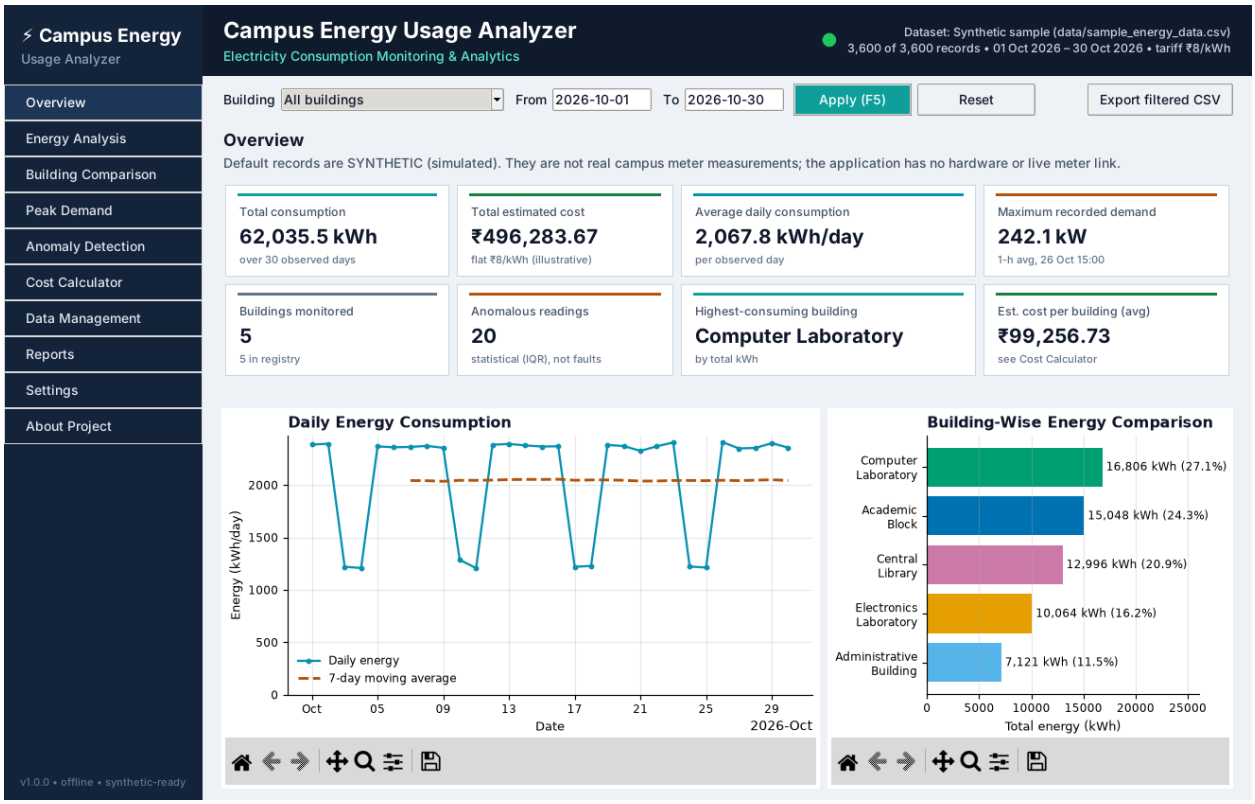



*Figure 4: Dashboard overview (actual screenshot of the running application)*

## 10. Results and Observations

All values below were computed by the application's own functions from data/sample_energy_data.csv (synthetic, seed 42) at the illustrative tariff of ₹ 8/kWh.

*Table 6: Campus-level results*

| Indicator | Value |
|---|---|
| Records / buildings / observed days | 3,600 / 5 / 30 |
| Total energy | 62,035.5 kWh |
| Average daily energy | 2,067.8 kWh/day |
| Average campus power | 86.16 kW |
| Maximum 1-hour average demand | 242.11 kW (Mon 26 Oct 15:00) |
| Campus load factor | 35.6% |
| Estimated cost (flat ₹ 8/kWh) | ₹496,283.67 |
| Statistical anomalies flagged | 20 |

*Table 7: Building-wise summary*

| Building | Energy (kWh) | Share | Avg kW | Peak kW | Load factor | Cost (₹) |
|---|---|---|---|---|---|---|
| Computer Laboratory | 16,806.5 | 27.1% | 23.34 | 84.00 | 27.8% | 134,452.00 |
| Academic Block | 15,048.5 | 24.3% | 20.90 | 62.68 | 33.3% | 120,387.70 |
| Central Library | 12,995.6 | 20.9% | 18.05 | 33.32 | 54.2% | 103,964.98 |
| Electronics Laboratory | 10,064.3 | 16.2% | 13.98 | 79.80 | 17.5% | 80,514.05 |
| Administrative Building | 7,120.6 | 11.5% | 9.89 | 30.39 | 32.5% | 56,964.95 |

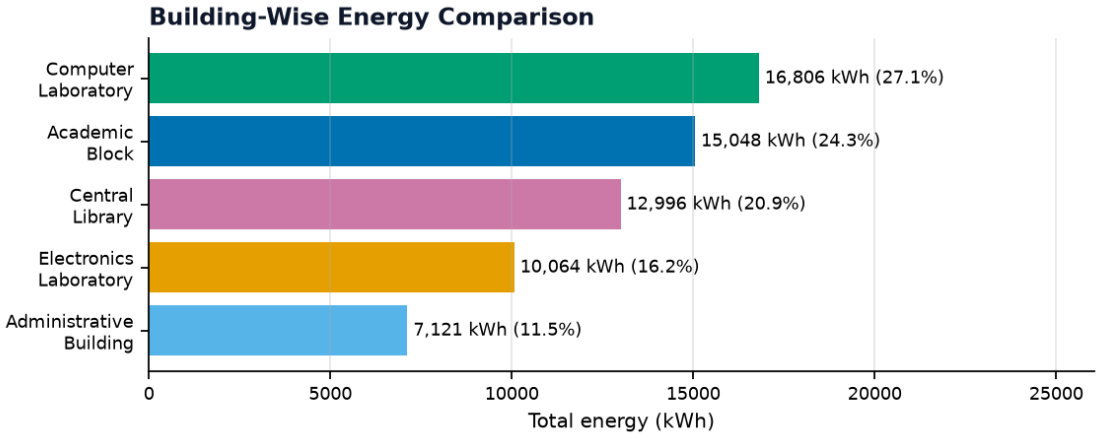



*Figure 5: Building-wise energy comparison*

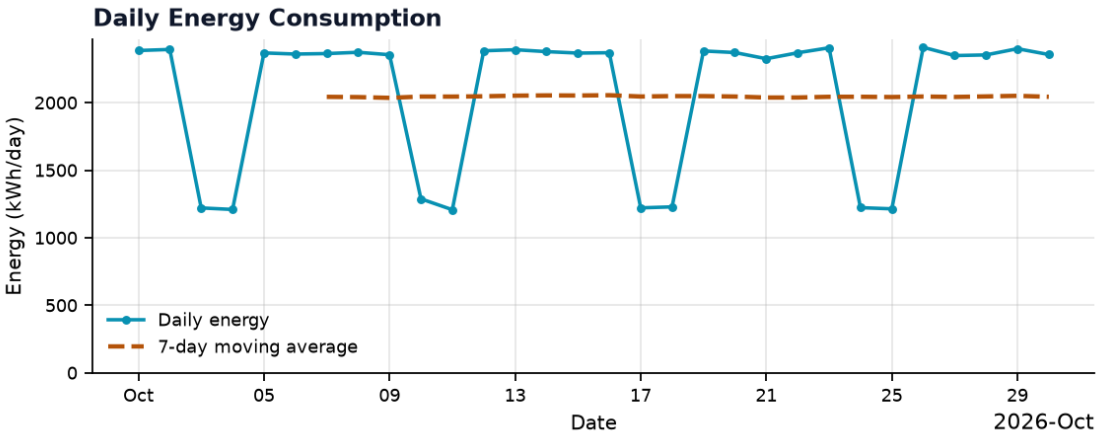



*Figure 6: Daily campus energy with 7-day moving average*

**Observations.** (i) Daily energy alternates between weekdays (mean 2,374 kWh) and weekends (mean 1,227 kWh); the dips in the daily chart are the eight weekend days. (ii) 5 of 5 buildings use less energy per day at weekends; the library changes least because it stays open. (iii) The Computer Laboratory has the highest total because of its standby load, although the Electronics Laboratory reaches a similar peak only during practical hours, giving it the lowest load factor (17.5%). (iv) All 150 building-days

contain the expected 24 readings, so the spikes and dips in the daily totals reflect recorded use, not missing data; on an incomplete dataset the application reports such days separately.

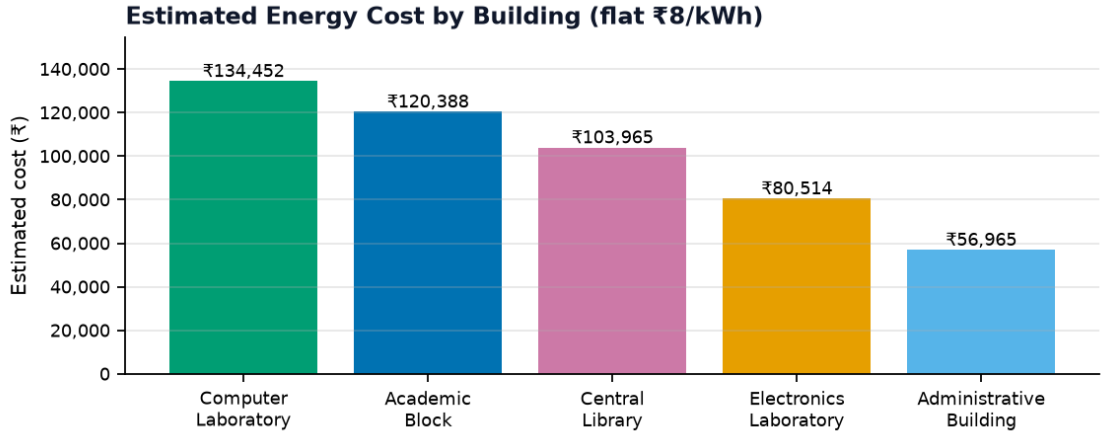



*Figure 7: Estimated cost by building (illustrative flat tariff)*

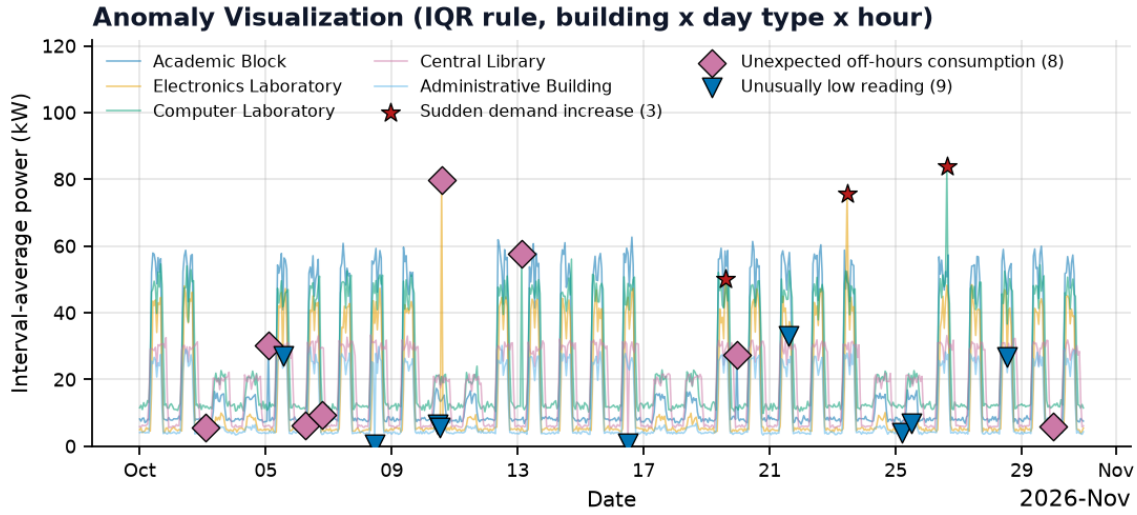



*Figure 8: Anomaly visualisation: flagged readings by category*

*Table 8: Evaluation of IQR grouping strategies against the 8 injected anomalies (k = 1.5)*

| Grouping strategy | Flags | Injected found | Other flags |
|---|---|---|---|
| Global campus threshold | 70 | 4 / 8 | 66 |
| Building-specific | 321 | 2 / 8 | 319 |
| Building x day type x hour | 20 | 8 / 8 | 12 |

A single campus threshold mostly flags whole high-load buildings and misses low readings, and a per-building threshold treats every busy afternoon as normal and every night as low; the building × day type × hour groups found all 8 injected anomalies with 12 additional flags. Flag categories: 9 unusually low reading, 8 unexpected off-hours consumption, 3 sudden demand increase. The additional flags are ordinary random variation that happens to fall outside the limits; this is the expected false-alarm behaviour of any fixed statistical rule. **A statistical anomaly is not proof of electrical equipment failure, energy theft, or malfunction.**

## 11. Testing and Validation

The suite was executed by this build script with *python -m pytest --cov=campus_energy*; results are read from the generated JUnit XML (docs/test_results.xml), not typed by hand. Total: **73 tests, 73 passed, 0 failed, 0 skipped** in 15.0 s; statement coverage of the package: 86%. GUI tests run the real Tkinter window (a virtual X display was used on Linux) and are skipped automatically if no display exists.

*Table 9: Actual automated test results*

| Test file | Scope | Pass | Fail | Skip |
|---|---|---|---|---|
| test_analytics.py | Sums, observed days, campus demand, peak, load factor, shares, % change, filters, moving average | 14 | 0 | 0 |
| test_anomalies.py | Quartiles, outlier mask, noise floor, constant data, small groups, injected anomalies, no fault claims | 8 | 0 | 0 |
| test_cost_calculator.py | Tariff multiplication, zero energy, invalid tariffs, tariff change, floating point | 11 | 0 | 0 |
| test_gui.py | Startup, navigation, filtering, invalid dates, empty state, exports, tariff, import, buildings, settings | 10 | 0 | 0 |
| test_simulator.py | Record count, seed reproducibility, non-negative values, E = P·t, timestamps, IDs, special methods | 10 | 0 | 0 |
| test_validation.py | Each rejection rule, no zero-filling, duplicates, ID/name conflicts, missing columns, empty file | 20 | 0 | 0 |

*Table 10: Validation of a deliberately faulty CSV file*

| Validation demo (data/sample_invalid_rows.csv) | Rows |
|---|---|
| Total rows read | 16 |
| Accepted | 5 |
| Rejected | 11 |
| Reason: building ID mapped to two names | 1 |
| Reason: duplicate building-timestamp combination | 1 |
| Reason: inconsistent power and energy (energy != power x interval) | 1 |
| Reason: infinite value in average_power_kw | 1 |
| Reason: infinite value in energy_kwh | 1 |
| Reason: invalid date format | 1 |
| Reason: invalid interval duration | 1 |
| Reason: invalid numerical value in average_power_kw | 1 |
| Reason: missing average_power_kw | 1 |
| Reason: missing building identifier | 1 |
| Reason: missing energy_kwh | 1 |
| Reason: missing timestamp | 1 |
| Reason: negative energy consumption | 1 |
| Reason: negative power value | 1 |

Manual checks: the application was launched, every page opened, all buttons exercised, and the screenshots in this report and in the repository were captured from the running program. Windows-specific behaviour (fonts, file dialogs) follows the same code path but was not executed on a Windows machine by the author of this build; it should be confirmed during the laboratory demonstration.

## 12. Discussion

The results confirm the expected behaviour of occupancy-driven loads: weekday daytime dominates, weekends fall sharply except in the library, and laboratories with short intensive sessions have low load factors. Because the data are simulated from stated profiles, these findings describe the model, not a real campus; their value is that the same pipeline will produce trustworthy figures for real CSV data. The anomaly evaluation shows why context matters: identical kW values can be normal at 14:00 and abnormal at 02:00.

## 13. Limitations

- Synthetic data only by default; no hardware, no live meter connection.
- Hourly averages hide short peaks; billing demand and power factor are not available.
- Flat-tariff costs exclude fixed, demand, tax and time-of-day components, so they are not bills.
- IQR flags depend on k, the noise floor and group size; a flag is not evidence of a fault.
- Building edits and deletions apply to the working dataset in memory and are not saved automatically.

## 14. Applications and Future Scope

- Energy-awareness dashboards for hostels, departments or small institutions using exported meter CSVs.
- Teaching aid for kW vs kWh, load factor and statistical outliers in ECE/EE laboratories.
- Future: 15-minute data and true billing-demand calculation; time-of-day tariffs; weather normalisation; SQLite storage of edits; Modbus/MQTT meter import; seasonal (rolling) baselines for anomaly detection.

## 15. Conclusion

A complete, tested and documented Python application was developed that turns interval electricity data into validated, explainable indicators, charts and reports. It processes 3,600 records for five buildings, implements eight analytical charts and a context-aware IQR detector that found all injected anomalies in the synthetic dataset, and passes its automated test suite. It is clear about what it cannot do: it does not measure real energy, reproduce bills or diagnose equipment faults.

## 16. References

[1] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.

[2] V. Chandola, A. Banerjee and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009.

[3] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357-362, 2020.

[4] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56-61.

[5] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[6] Bureau of Energy Efficiency, *Energy Conservation Building Code 2017*. New Delhi: Ministry of Power, Government of India, 2017.

[7] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python 3 documentation, docs.python.org.

[8] ReportLab Inc., *ReportLab PDF Library User Guide*, docs.reportlab.com.

[9] pytest developers, *pytest documentation*, docs.pytest.org.

## Appendix A: Installation and Execution

Source code, tests, sample data and documentation: **https://github.com/CypherCodeNinjaX/campus-energy-usage-analyzer** (folder src/campus_energy). Windows PowerShell, from the folder that should contain the project:

```
git clone https://github.com/CypherCodeNinjaX/campus-energy-usage-analyzer.git
cd campus-energy-usage-analyzer
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m campus_energy.main   # start the application
.venv\Scripts\python.exe -m pytest             # run the tests
```

Calling .venv\Scripts\python.exe directly works even when PowerShell's execution policy blocks Activate.ps1. requirements.txt installs the project itself in editable mode (-e .), which is what makes *python -m campus_energy.main* work from any folder. CSV import expects the header `timestamp,building_id,building_name,interval_hours,average_power_kw,energy_kwh`.

## Appendix B: Team Contributions

*Table 11: To be completed by the group (maximum five members)*

| Member (name / roll no.) | Module(s) owned | Individual contribution |
|---|---|---|
| | | |
| | | |
| | | |
| | | |